COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/31/2026	**DATE SUBMITTED:**	08/05/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

```cpp
#include <iostream>
#include <stack>

int main(){

    std::stack<int> stack1;
    std::stack<int> stack2;

    std::cout << "Testing the stack STL" << std::endl;

    // isEmpty
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;

    // push
    stack1.push(10);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(9);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(8);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    stack1.push(7);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;

    // pop
    stack1.pop();
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    std::cout << "Is the stack empty? " << stack1.empty() << std::endl;
    std::cout << "THe size of the stack is " << stack1.size() << std::endl;

    // emplace
    stack1.emplace(11);
    std::cout << "The top of the stack is " << stack1.top() << std::endl;
    std::cout << "THe size of the stack is " << stack1.size() << std::endl;

    // swap
    stack2.push(12);
    stack1.swap(stack2);
    std::cout << "The top of the stack1 is " << stack1.top() << std::endl;
    std::cout << "The top of the stack2 is " << stack2.top() << std::endl;

    return 0;
}
```

FIgure 1. Code of the Program Stacks Implementation Using STL.

This implementation uses the STL (Standard Template Library) method to perform push, emplace, swap, and pop operations on stacks. isEmpty checks if the stack is empty. Push adds the value on top of the stack. Pop

removes the top value of the stack. Emplace performs similarly by adding the value on top of the stack as well. Swap changes the values from one stack to another.

```
Testing the stack STL
Is the stack empty? 1
The top of the stack is 10
The top of the stack is 9
The top of the stack is 8
The top of the stack is 7
The top of the stack is 8
Is the stack empty? 0
THe size of the stack is 3
The top of the stack is 11
THe size of the stack is 4
The top of the stack1 is 12
The top of the stack2 is 11
```

FIgure 2. Console Output of the Program Stacks Implementation Using STL.

This console output contains the various operations performed in the stack using the STL method. The empty stack checks output in 1 or 0, which means true or false. As seen in the figure, the top of the stack goes down from 10 to 7 because of the values added on top of the list. The list then goes back to 8 because of the value 7 being popped. The size of the stack has changed since one element in the stack has popped. Since emplace() does a similar process to push(), the size of the stack and current top value also change.

ILO B1

```
#include <iostream>

// global declaration
#define maxCap 100
int stackArr[maxCap];
int top = -1, newData;

// prototype function
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
```

```cpp
int main(){
    // main driver code
    int choice;

    while (true){
        std::cout << "================================================" << std::endl;
        std::cout << "Stack Operations\n";
        std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull" << std::endl;
        std::cout << "================================================" << std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << "Is the stack empty? " << isEmpty() << std::endl;
            break;
            case 5: std::cout << "Is the stack full? " << isFull() << std::endl;
            break;
            default: std::cout << "Invalid choice" << std::endl;
            break;
        }
    }

}
```

```
45  // function definition
46  bool isEmpty(){
47      // check the top if it is -1
48      if(top == -1) return true;
49      return false;
50  }
51
52  bool isFull(){
53      // how do we verify if the stack is full
54      if(top == maxCap - 1) return true;
55      return false;
56  }
57
58  void push(){
59      // error checking
60      if(isFull()){
61          std::cout << "Stack Overflow" << std::endl;
62          return;
63      }
64
65      // pushing to the stack
66      std::cout << "Enter a new value" << std::endl;
67      std::cin >> newData;
68
69      // how do we insert the data into the stack?
70      stackArr[++top] = newData;
71  }
```

```
73  void pop(){
74      // errpr checking
75      if(isEmpty()){
76          std::cout << "Stack Underflow" << std::endl;
77          return;
78      }
79
80      // DIsplay the value that we are going to pop
81      std::cout << "Popping: " << stackArr[top] << std::endl;
82
83      // decrement the value from the stack
84      top--;
85
86  }
87
88  void Top(){
89      // error catching
90      if(isEmpty()){
91          std::cout << "The stack is empty" << std::endl;
92          return;
93      }
94
95      // check the top element
96      std::cout << "Top Element: " << stackArr[top] << std::endl;
97
98  }
```

FIgure 1. Driver Code of the Program Stacks Using Arrays.

This code contains the implementation of stacks using arrays, where this program contains the standard structure with a global declaration, function prototypes, main function, and function definitions. Instead of using an STL library, the actions of managing stacks are written directly in the program code. This includes the isEmpty, isFull, push, pop, and Top functions, which are used in a user input style menu. The array contains a maximum cap, stack array, and a -1 to state an empty stack.

```
==========================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==========================================
1
Enter a new value
1
==========================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==========================================
1
Enter a new value
2
==========================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==========================================
1
Enter a new value
3
```

```
==========================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==========================================
4
Is the stack empty? 0
==========================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==========================================
5
Is the stack full? 0
==========================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull
==========================================
```

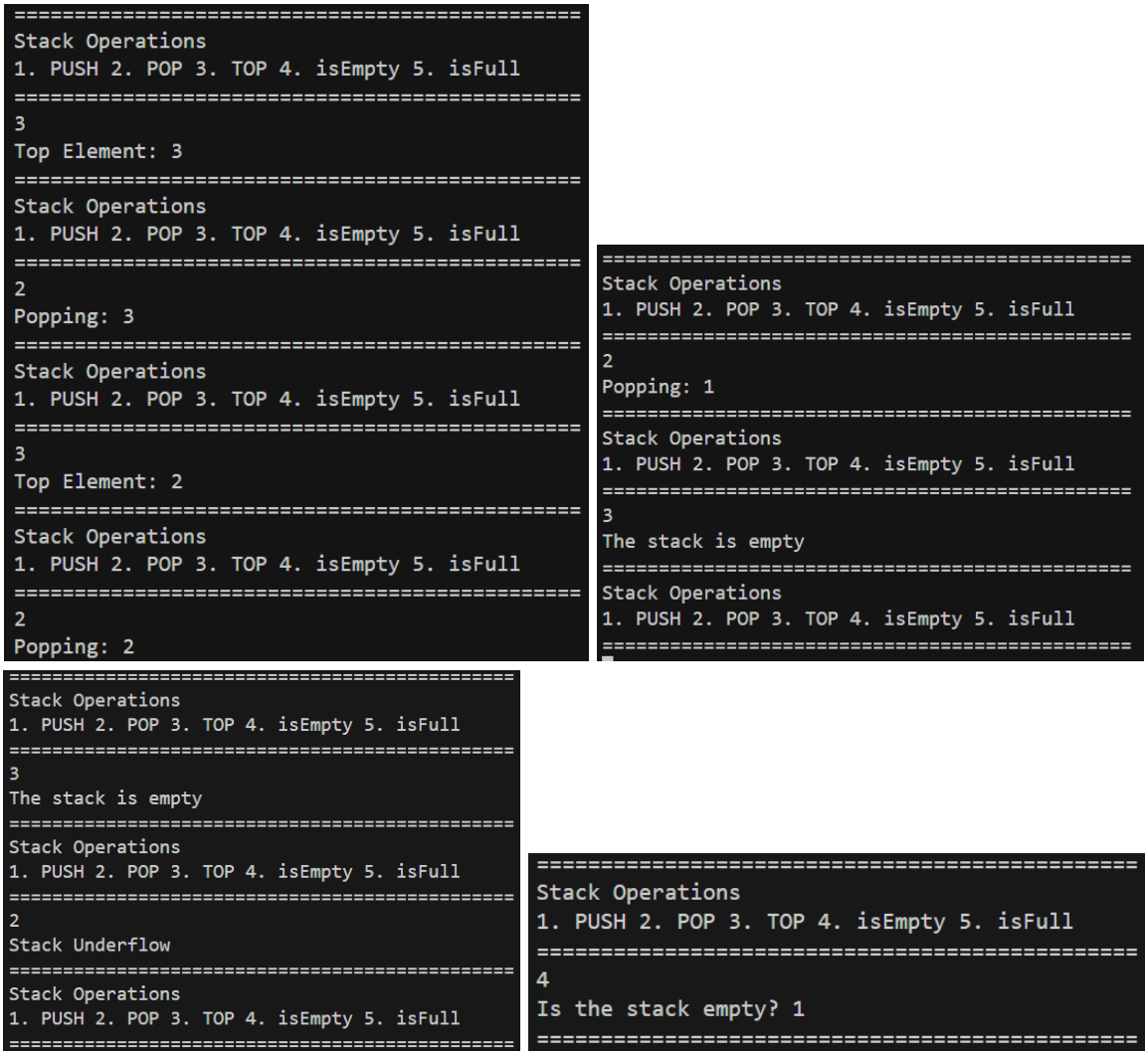


FIgure 2. Console Output of the Program Stacks Using Arrays.

This figure contains the console output of the array implementation of the stack program, which has five options to select push, pop, top, isEmpty, and isFull on various values of integers added to the stack array. The program has checks for empty stacks, stack underflow, and stack overflow. It also has the standard stack actions, such as push, pop, and top. The menu and a high number of maximum stack numbers allow for a continuous pushing and popping of values in a stack.

ILO B2

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

// class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;     // VALUE THAT WE NEED TO STORE
    Node* next; // POINTER TO THE NEXT NODE
};

// global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

// operations for the stack

// push
template <typename T>
void push(T newData){
    // create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;

    // new node points to the current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}
```

```cpp
// pop
template <typename T>
T pop(){
    //check if the stack is empty:
    if(head<T> == nullptr){
        std::cout << "stack underflow. " << std::endl;
        return T();
    }

    // save the current top node
    Node<T>* temp = head<T>;
    // store its data
    T tempVal = temp->data;

    // move the head to the next node
    head<T> = head<T>->next;
    // free the removed node
    delete temp;
    // return the removed value
    return tempVal;
}
```

```cpp
// isEmpty
template <typename T>
bool isEmpty(){
    return head <T> == nullptr;
}

// top
template<typename T>
void Top(){
    // error catching
    if(isEmpty<T>()){
        std::cout << "Stack underflow " << std::endl;
        return;
    }
    // display the Top node
    std::cout << "Top of Stack: " << head<T>->data << std::endl;
}
```

```
69   // display All
70   template<typename T>
71   void displayALL(){
72       if(head<T> == nullptr){
73           std::cout << "Stack underflow\n";
74           return;
75       }
76       std::cout << "Displaying the stack: \n";
77       Node<T>* temp = head<T>;
78       while(temp != nullptr){
79           std::cout << temp->data << "\n";
80           temp = temp->next;
81       }
82       std::cout << "\n";
83   }
84   #endif
```

FIgure 1. Header File of the Program Stacks Using Linked Lists.

This figure contains the header file of the linked list implementation of a stacks program, which uses the function templates containing the push, pop, isEmpty, top, and displayALL functions to perform the stack operations. This header file is used in the next figure, where the characters c, p, and e are pushed to a stack list. This allows for the program in the next figure to occupy less lines of code, and also for a reusable code when used in other tasks..

```
1   #include <iostream>
2   #include "stackList.h"
3
4   int main(){
5
6       std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";
7       // push
8       push('c');
9       push('p');
10      push('e');
11
12      // display the current top
13      Top<char>(); // Top should be 'e'
14      displayALL<char>();
15      std::cout << "\n ------ Popping all the elements ------ \n";
16
17      while(!isEmpty<char>()){
18          std::cout << "Popped: " << pop<char>() << std::endl;
19      }
20
21      std::cout << "\n ------ Testing the Underflow ------ \n";
22      pop<char>();
23
24      return 0;
25  }
```

FIgure 2. Driver Code of the Program Stacks Using Linked Lists.

This figure contains the driver code of the program using linked lists, which contains a demonstration of the push, pop, and stack underflow of the stacks. The characters of cpe are pushed individually to populate the stack. Then, all the elements are popped in a while loop with an isEmpty check. After all elements are popped,

an underflow check is used to test the underflow.

```
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of Stack: e
Displaying the stack:
e
p
c


 ------ Popping all the elements ------
Popped: e
Popped: p
Popped: c

 ------ Testing the Underflow ------
```

FIgure 3. Console Output of the Program Stacks Using Linked Lists.

This figure contains the console output of the program using the linked list implementation to insert the individual characters into the linked list stack. The top of the stack contains the top character of the stack, which is e. A display all function is used to display the stack row by row. When popping, the console displays the individual elements being popped. It is done backwards because the characters are inputted sequentially before, so the popping goes the other way around.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C

Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	`Expression: (A+B)+(C-D)` `Valid: Y`	There are proper opening and closing parenthesis, so it is **valid**.
((A+B)+(C-D)	N	`Expression: ((A+B)+(C-D)` `Valid: N`	There is an excess opening parenthesis at the first term, so it is **invalid**.
((A+B)+[C-D])	Y	`Expression: ((A+B)+[C-D])` `Valid: Y`	There is a proper opening and closing bracket, so it is **valid**.
((A+B]+[C-D]}	N	`Expression: ((A+B]+[C-D]}` `Valid: N`	The expression has a missing bracket, so it is **invalid**.

```cpp
#include <iostream>
#include <string>

// global declaration
#define maxCap 100
char stackArr[maxCap];
int top = -1;

// prototype function
bool isEmpty();
bool isFull();
void push();
char pop();
bool match();
bool balanceSign(std::string);

// main driver function
int main(){
    std::string test[] = {"(A+B)+(C-D)", "((A+B)+(C-D)", "((A+B)+[C-D])", "((A+B]+[C-D])"};

    for(int i = 0; i < 4; i++){
        top = -1;

        std::cout << "Expression: " << test[i] << std::endl;

        if(balanceSign(test[i])){
            std::cout << "Valid: Y";
        } else {
            std::cout << "Valid: N";
        }
        std::cout << std::endl;
    }

    return 0;
}
```

```cpp
// FUNCTION DEFINITION
bool isEmpty(){
    if(top == -1) return true;
    return false;
}

bool isFull(){
    if(top == maxCap - 1) return true;
    return false;
}

void push(char value){
    if(isFull()){
        std::cout << "Stack Overflow\n";
        return;
    }

    stackArr[++top] = value;
}

char pop(){
    if(isEmpty()){
        return ' ';
    }

    return stackArr[top--];
}
```

```cpp
bool match(char open, char close){
    if(open == '(' && close == ')'){
        return true;
    }
    if(open == '[' && close == ']'){
        return true;
    }
    if(open == '{' && close == '}'){
        return true;
    }
    return false;
}

bool balanceSign(std::string expression){
    for(int i = 0; i < expression.length(); i++){
        char ch = expression[i];

        if(ch == '(' || ch == '[' || ch == '{'){
            push(ch);
        }
        else if(ch == ')' || ch == ']' || ch == '}'){
            if(isEmpty()){
                return false;
            }
            char popChar = pop();

            if(!match(popChar, ch)){
                return false;
            }
        }
    }

    if(isEmpty()){
        return true;
    }
    return false;
}
```

Figure 1. Code of Stacks Using Arrays.

This figure contains the array implementation of the stack using a similar structure to the previous figures in the procedure section. The difference comes from the implementation of a balance symbol checker function in the code, which contains the checks for the brackets, parentheses, and curly brackets. The first if statement checks if the characters contain either the proper front and back brackets, parentheses, and curly brackets. Otherwise, it returns a false statement. This demonstrates the application of stacks to solving problems, such as determining if the symbols are balanced or not.

```cpp
#include <iostream>
#include <string>
#include "stackList.h"

// function to balance sign
bool balanceSign(std::string expression){
    head<char> = nullptr;

    for(int i = 0; i < expression.length(); i++){
        char ch = expression[i];

        if(ch == '(' || ch == '[' || ch == '{'){
            push(ch);
        }
        else if(ch == ')' || ch == ']' || ch == '}'){
            if(isEmpty<char>()){
                return false;
            }
            char popChar = pop<char>();

            if(!match(popChar, ch)){
                return false;
            }
        }
    }

    return isEmpty<char>();
}
```

```cpp
// main
int main(){
    std::string test[] = {"(A+B)+(C-D)", "((A+B)+(C-D)", "((A+B)+[C-D])", "((A+B]+[C-D]}"};

    for(int i = 0; i < 4; i++){
        std::cout << test[i] << std::endl;

        if(balanceSign(test[i])){
            std::cout << "Valid: Y\n";
        } else {
            std::cout << "Valid: N\n";
        }
    }

    return 0;
}
```

Figure 2. Code of Stacks Using Linked Lists.

This figure contains the implementation of linked lists for performing stack operations. The difference in this code compared to the previous code contains a similar if, else, and if statement containing the front and back bracket characters for checking the balance of the strings. The difference comes from the match function for the characters used in the header file to match the opening and closing symbols.

```
85  // additional function to match symbol
86  template<typename T>
87  bool match(T open, T close){
88
89      if(open == '(' && close == ')'){
90          return true;
91      }
92      if(open == '[' && close == ']'){
93          return true;
94      }
95      if(open == '{' && close == '}'){
96          return true;
97      }
98      return false;
99  }
00
01  #endif
```

Figure 3. Additional Function in stackList.h to Match Symbols.

This additional function contains the function to match the opening and closing brace symbols for the previous file, which contains the characters for parentheses, brackets, and curly braces. This is used by the code in the previous figure to check if the character matches for the opening and closing brackets.

```
1   #include <iostream>
2   #include <stack>
3   #include <string>
4
5   // function prototype
6   bool match();
7   bool balanceSign(std::string);
8
9   // main function
10  int main(){
11      std::string test[] = {"(A+B)+(C-D)", "((A+B)+(C-D)", "((A+B)+[C-D])", "((A+B]+[C-D])"};
12
13      for(int i = 0; i < 4; i++){
14          std::cout << "Expression: " << test[i] << std::endl;
15
16          if(balanceSign(test[i])){
17              std::cout << "Valid: Y";
18          } else {
19              std::cout << "Valid: N";
20          }
21          std::cout << std::endl;
22      }
23
24      return 0;
25  }
26
27  // function to match symbols
28  bool match(char open, char close){
29      if(open == '(' && close == ')')
30          return true;
31      if(open == '[' && close == ']')
32          return true;
33      if(open == '{' && close == '}')
34          return true;
35
36      return false;
37  }
```

```cpp
// function to see whether expression is balanced
bool balanceSign(std::string expression){
    std::stack<char> sign;

    for(int i = 0; i < expression.length(); i++){
        char ch = expression[i];

        if(ch == '(' || ch == '[' || ch == '{'){
            sign.push(ch);
        }
        else if(ch == ')' || ch == ']' || ch == '}'){
            if(sign.empty())
                return false;

            char popChar = sign.top();
            sign.pop();

            if(!match(popChar, ch))
                return false;
        }
    }

    if(sign.empty())
        return true;

    return false;
}
```

Figure 4. Code of Stacks Using STL.

This figure contains the STL implementation of the stack operation program, which uses the included stack library in C++. The program uses the STL library to perform the push and pop stack operations for the sign balance check. The results for each implementation are the same, which means they all have the same answer for the valid and invalid checks in the opening and closing braces for each expression tested.

CONCLUSION

Overall, this hands-on activity has demonstrated the use of stacks in various implementations, such as using the STL (Standard Template Library), header file of linked lists functions, and the arrays method. Each implementation has their own traits and behaviors in writing and running the program. Arrays have a maximum cap in the number of slots to add in the stack before overflowing. Linked lists traverse through the data or symbols in a linear method. Using the stack library simply imports a library from the C++ Standard Template Library to perform the basic pop, push, and top operations in stacks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Clarification and Examples of Concepts DIscussed in the Activity**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **THe AI was used to make clearer examples of the concepts in the discussion of this activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.